🤖 NewsBot Intelligence System

ITAI 2373 - Mid-Term Group Project Template

Team Members: [Lufei Yu & Richard Cortez] **Date:** [04/01/2026] **GitHub Repository:** [https://github.com/Lufei-Yu/ITAI-2373-Natural-Language-Processing/tree/main/ITAI2373-NewsBot-Midterm]

🎯 Project Overview

Welcome to your NewsBot Intelligence System! This notebook will guide you through building a comprehensive NLP system that:

- 📰 **Processes** news articles with advanced text cleaning
- 🏷️ **Classifies** articles into categories (Politics, Sports, Technology, Business, Entertainment, Health)
- 🔍 **Extracts** named entities (people, organizations, locations, dates, money)
- 😊 **Analyzes** sentiment and emotional tone
- 📊 **Generates** insights for business intelligence

📚 Module Integration Checklist

- ☐ **Module 1:** NLP applications and real-world context
- ☐ **Module 2:** Text preprocessing pipeline
- ☐ **Module 3:** TF-IDF feature extraction
- ☐ **Module 4:** POS tagging analysis
- ☐ **Module 5:** Syntax parsing and semantic analysis
- ☐ **Module 6:** Sentiment and emotion analysis
- ☐ **Module 7:** Text classification system
- ☐ **Module 8:** Named Entity Recognition

📦 Setup and Installation

Let's start by installing and importing all the libraries we'll need for our NewsBot system.

```
# Install required packages (run this cell first!)
!pip install spacy scikit-learn nltk pandas matplotlib seaborn wordcloud plotly
```

```
!python -m spacy download en_core_web_sm

# Download NLTK data
import nltk
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('vader_lexicon')
nltk.download('averaged_perceptron_tagger')

print("✅ All packages installed successfully!")
```

```
Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: wordcloud in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/p
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/p
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/pyt
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.1
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3
Requirement already satisfied: thinc<8.4.0,>=8.3.12 in /usr/local/lib/python3.
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.
Requirement already satisfied: srsly<3.0.0,>=2.5.3 in /usr/local/lib/python3.1
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/pytho
Requirement already satisfied: weasel<2.0.0,>=1.0.0 in /usr/local/lib/python3.
Requirement already satisfied: confection<2.0.0,>=1.3.2 in /usr/local/lib/pyth
Requirement already satisfied: typer<1.0.0,>=0.3.0 in /usr/local/lib/python3.1
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.1
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/pytho
Requirement already satisfied: pydantic<3.0.0,>=2.0.0 in /usr/local/lib/python
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/di
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dis
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python
```

```
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/pyt
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/pyth
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/pyth
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.12
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.
Requirement already satisfied: cloudpathlib>=0.7.0 in /usr/local/lib/python3.1
Requirement already satisfied: smart-open>=5.2.1 in /usr/local/lib/python3.12/
```

```
# Import all necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
import plotly.express as px
import plotly.graph_objects as go
from collections import Counter, defaultdict
import re
import warnings
warnings.filterwarnings('ignore')

# NLP Libraries
import spacy
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.stem import WordNetLemmatizer
from nltk.sentiment import SentimentIntensityAnalyzer
from nltk.tag import pos_tag

# Scikit-learn for machine learning
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, accuracy_s
from sklearn.pipeline import Pipeline

# Load spaCy model
nlp = spacy.load('en_core_web_sm')

# Set up plotting style
plt.style.use('default')
sns.set_palette("husl")
```

```
print("📚 All libraries imported successfully!")
print(f"🔧 spaCy model loaded: {nlp.meta['name']} v{nlp.meta['version']}")
```

```
📚 All libraries imported successfully!
🔧 spaCy model loaded: core_web_sm v3.8.0
```

📊 Data Loading and Exploration

🎯 Module 1: Understanding Our NLP Application

Before we dive into the technical implementation, let's understand the real-world context of our NewsBot Intelligence System. This system addresses several business needs:

1. **Media Monitoring:** Automatically categorize and track news coverage
2. **Business Intelligence:** Extract key entities and sentiment trends
3. **Content Management:** Organize large volumes of news content
4. **Market Research:** Understand public sentiment about topics and entities

💡 **Discussion Question:** What other real-world applications can you think of for this type of system? Consider different industries and use cases.

```
# Load your dataset
# 💡 TIP: If using the provided dataset, upload it to Colab first
# 💡 TIP: You can also use sample datasets like BBC News or 20 Newsgroups

# Option 1: Load provided dataset
# df = pd.read_csv('news_dataset.csv')

# Option 2: Load BBC News dataset (if using alternative)
# You can download this from: https://www.kaggle.com/c/learn-ai-bbc/data

# Option 3: Create sample data for testing (remove this when you have real data
sample_data = {
    'article_id': range(1, 11),
    'title': [
        'Apple Inc. Reports Record Quarterly Earnings',
        'Manchester United Defeats Chelsea 3-1',
        'New AI Technology Revolutionizes Healthcare',
        'President Biden Announces Climate Initiative',
        'Netflix Releases New Original Series',
        'Tesla Stock Rises After Production Update',
        'Olympic Games Begin in Paris',
        'Google Launches New Search Algorithm',
        'Congress Passes Infrastructure Bill',
        'Disney+ Subscriber Numbers Grow'
    ],
```

```
    'content': [
        'Apple Inc. announced record quarterly earnings today, with CEO Tim Coc
        'Manchester United secured a convincing 3-1 victory over Chelsea at Olc
        'A breakthrough AI system developed by researchers at Stanford Universi
        'President Joe Biden unveiled a comprehensive climate change initiative
        'Netflix premiered its latest original series to critical acclaim, feat
        'Tesla shares jumped 8% in after-hours trading following the company\'s
        'The 2024 Olympic Games officially began in Paris with a spectacular op
        'Google introduced a new search algorithm that promises more accurate a
        'The U.S. Congress passed a bipartisan infrastructure bill allocating $
        'Disney+ reported strong subscriber growth in Q3, reaching 150 million
    ],
    'category': ['Business', 'Sports', 'Technology', 'Politics', 'Entertainment
                 'Business', 'Sports', 'Technology', 'Politics', 'Entertainment'
    'date': ['2024-01-15'] * 10,
    'source': ['TechNews', 'SportsTimes', 'TechDaily', 'PoliticsToday', 'Entert
               'BusinessWire', 'SportsCentral', 'TechReview', 'NewsNow', 'ShowBi
}

df = pd.DataFrame(sample_data)

print(f"📊 Dataset loaded successfully!")
print(f"📈 Shape: {df.shape}")
print(f"📋 Columns: {list(df.columns)}")

# Display first few rows
df.head()
```

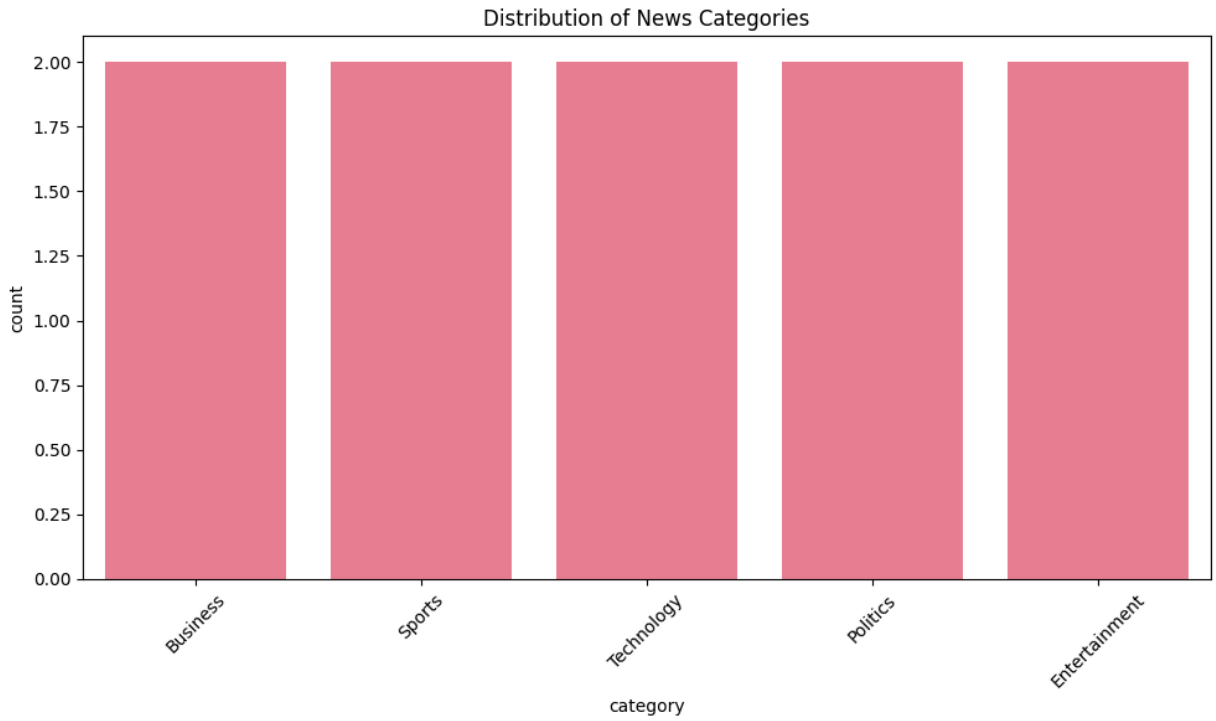
```
📊 Dataset loaded successfully!
📈 Shape: (10, 6)
📋 Columns: ['article_id', 'title', 'content', 'category', 'date', 'source']
```

	article_id	title	content	category	date	source
0	1	Apple Inc. Reports Record Quarterly Earnings	Apple Inc. announced record quarterly earnings...	Business	2024-01-15	TechNews
1	2	Manchester United Defeats	Manchester United secured a convincing 3-	Sports	2024-01-15	SportsTimes

```
# Basic dataset exploration
print("📊 DATASET OVERVIEW")
print("=" * 50)
print(f"Total articles: {len(df)}")
print(f"Unique categories: {df['category'].nunique()}")
print(f"Categories: {df['category'].unique().tolist()}")
print(f"Date range: {df['date'].min()} to {df['date'].max()}")
print(f"Unique sources: {df['source'].nunique()}")
```

```
print("\n📈 CATEGORY DISTRIBUTION")
print("=" * 50)
category_counts = df['category'].value_counts()
print(category_counts)

# Visualize category distribution
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x='category', order=category_counts.index)
plt.title('Distribution of News Categories')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# 💡 STUDENT TASK: Add your own exploratory analysis here
# - Check for missing values
# - Analyze text length distribution
# - Examine source distribution
# - Look for any data quality issues
```

```
📊 DATASET OVERVIEW
==================================================
Total articles: 10
Unique categories: 5
Categories: ['Business', 'Sports', 'Technology', 'Politics', 'Entertainment']
Date range: 2024-01-15 to 2024-01-15
Unique sources: 10

📈 CATEGORY DISTRIBUTION
==================================================
category
Business         2
Sports           2
Technology       2
Politics         2
Entertainment    2
Name: count, dtype: int64
```



🧹 Text Preprocessing Pipeline

🎯 Module 2: Advanced Text Preprocessing

Now we'll implement a comprehensive text preprocessing pipeline that cleans and normalizes our news articles. This is crucial for all downstream NLP tasks.

Key Preprocessing Steps:

1. **Text Cleaning:** Remove HTML, URLs, special characters
2. **Tokenization:** Split text into individual words
3. **Normalization:** Convert to lowercase, handle contractions
4. **Stop Word Removal:** Remove common words that don't carry meaning
5. **Lemmatization:** Reduce words to their base form

💡 **Think About:** Why is preprocessing so important? What happens if we skip these steps?

```
# Initialize preprocessing tools
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

# Download the missing punkt_tab resource
nltk.download('punkt_tab', quiet=True)

def clean_text(text):
    """
    Comprehensive text cleaning function

    💡 TIP: This function should handle:
    - HTML tags and entities
    - URLs and email addresses
    - Special characters and numbers
    - Extra whitespace
    """
    if pd.isna(text):
        return ""

    # Convert to string and lowercase
    text = str(text).lower()

    # 🚀 YOUR CODE HERE: Implement text cleaning
    # Remove HTML tags
    text = re.sub(r'<[^>]+>', '', text)

    # Remove URLs
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)

    # Remove email addresses
    text = re.sub(r'\S+@\S+', '', text)
```

```
    # Remove special characters and digits (keep only letters and spaces)
    text = re.sub(r'[^a-zA-Z\s]', '', text)

    # Remove extra whitespace
    text = re.sub(r'\s+', ' ', text).strip()

    return text

def preprocess_text(text, remove_stopwords=True, lemmatize=True):
    """
    Complete preprocessing pipeline

    💡 TIP: This function should:
    - Clean the text
    - Tokenize into words
    - Remove stop words (optional)
    - Lemmatize words (optional)
    - Return processed text
    """
    # Clean text
    text = clean_text(text)

    if not text:
        return ""

    # 🚀 YOUR CODE HERE: Implement tokenization and preprocessing
    # Tokenize
    tokens = word_tokenize(text)

    # Remove stop words if requested
    if remove_stopwords:
        tokens = [token for token in tokens if token not in stop_words]

    # Lemmatize if requested
    if lemmatize:
        tokens = [lemmatizer.lemmatize(token) for token in tokens]

    # Filter out very short words
    tokens = [token for token in tokens if len(token) > 2]

    return ' '.join(tokens)

# Test the preprocessing function
sample_text = "Apple Inc. announced record quarterly earnings today! Visit http
print("Original text:")
print(sample_text)
print("\nCleaned text:")
print(clean_text(sample_text))
print("\nFully preprocessed text:")
print(preprocess_text(sample_text))
```

```
Original text:
Apple Inc. announced record quarterly earnings today! Visit https://apple.com fc

Cleaned text:
apple inc announced record quarterly earnings today visit for more info technews

Fully preprocessed text:
apple inc announced record quarterly earnings today visit info technews
```

```
# Apply preprocessing to the dataset
print("🧹 Preprocessing all articles...")

# Create new columns for processed text
df['title_clean'] = df['title'].apply(clean_text)
df['content_clean'] = df['content'].apply(clean_text)
df['title_processed'] = df['title'].apply(preprocess_text)
df['content_processed'] = df['content'].apply(preprocess_text)

# Combine title and content for full article analysis
df['full_text'] = df['title'] + ' ' + df['content']
df['full_text_processed'] = df['full_text'].apply(preprocess_text)

print("✅ Preprocessing complete!")

# Show before and after examples
print("\n📝 BEFORE AND AFTER EXAMPLES")
print("=" * 60)
for i in range(min(3, len(df))):
    print(f"\nExample {i+1}:")
    print(f"Original: {df.iloc[i]['full_text'][:100]}...")
    print(f"Processed: {df.iloc[i]['full_text_processed'][:100]}...")

# 💡 STUDENT TASK: Analyze the preprocessing results
# - Calculate average text length before and after
# - Count unique words before and after
# - Identify the most common words after preprocessing
```

```
🧹 Preprocessing all articles...
✅ Preprocessing complete!

📝 BEFORE AND AFTER EXAMPLES
============================================================

Example 1:
Original: Apple Inc. Reports Record Quarterly Earnings Apple Inc. announced recc
Processed: apple inc report record quarterly earnings apple inc announced record

Example 2:
Original: Manchester United Defeats Chelsea 3-1 Manchester United secured a conv
Processed: manchester united defeat chelsea manchester united secured convincing

Example 3:
```

```
Original: New AI Technology Revolutionizes Healthcare A breakthrough AI system d
Processed: new technology revolutionizes healthcare breakthrough system develope
```

📊 Feature Extraction and Statistical Analysis

🎯 Module 3: TF-IDF Analysis

Now we'll extract numerical features from our text using TF-IDF (Term Frequency-Inverse Document Frequency). This technique helps us identify the most important words in each document and across the entire corpus.

TF-IDF Key Concepts:

- **Term Frequency (TF):** How often a word appears in a document
- **Inverse Document Frequency (IDF):** How rare a word is across all documents
- **TF-IDF Score:** TF × IDF - balances frequency with uniqueness

💡 **Business Value:** TF-IDF helps us identify the most distinctive and important terms for each news category.

```
# Create TF-IDF vectorizer
# 💡 TIP: Experiment with different parameters:
# - max_features: limit vocabulary size
# - ngram_range: include phrases (1,1) for words, (1,2) for words+bigrams
# - min_df: ignore terms that appear in less than min_df documents
# - max_df: ignore terms that appear in more than max_df fraction of documents

tfidf_vectorizer = TfidfVectorizer(
    max_features=5000,  # Limit vocabulary for computational efficiency
    ngram_range=(1, 2),  # Include unigrams and bigrams
    min_df=2,  # Ignore terms that appear in less than 2 documents
    max_df=0.8  # Ignore terms that appear in more than 80% of documents
)

# Fit and transform the processed text
print("🔢 Creating TF-IDF features...")
tfidf_matrix = tfidf_vectorizer.fit_transform(df['full_text_processed'])
feature_names = tfidf_vectorizer.get_feature_names_out()

print(f"✅ TF-IDF matrix created!")
print(f"📊 Shape: {tfidf_matrix.shape}")
print(f"📝 Vocabulary size: {len(feature_names)}")
print(f"🔢 Sparsity: {(1 - tfidf_matrix.nnz / (tfidf_matrix.shape[0] * tfidf_m

# Convert to DataFrame for easier analysis
tfidf_df = pd.DataFrame(tfidf_matrix.toarray(), columns=feature_names)
tfidf_df['category'] = df['category'].values
```

```
print("\n🔍 Sample TF-IDF features:")
print(tfidf_df.iloc[:3, :10])  # Show first 3 rows and 10 features
```

```
🔢 Creating TF-IDF features...
✅ TF-IDF matrix created!
📊 Shape: (10, 4)
📝 Vocabulary size: 4
🔢 Sparsity: 77.50%

🔍 Sample TF-IDF features:
     growth       new   promise    strong    category
0  0.707107  0.000000  0.000000  0.707107    Business
1  0.000000  0.000000  0.000000  0.000000      Sports
2  0.000000  0.658454  0.752621  0.000000  Technology
```

```
# Analyze most important terms per category
def get_top_tfidf_terms(category, n_terms=10):
    """
    Get top TF-IDF terms for a specific category

    💡 TIP: This function should:
    - Filter data for the specific category
    - Calculate mean TF-IDF scores for each term
    - Return top N terms with highest scores
    """
    # 🚀 YOUR CODE HERE: Implement category-specific TF-IDF analysis
    category_data = tfidf_df[tfidf_df['category'] == category]

    # Calculate mean TF-IDF scores for this category (excluding the category cc
    mean_scores = category_data.drop('category', axis=1).mean().sort_values(asc

    return mean_scores.head(n_terms)

# Analyze top terms for each category
print("🏷️ TOP TF-IDF TERMS BY CATEGORY")
print("=" * 50)

categories = df['category'].unique()
category_terms = {}

for category in categories:
    top_terms = get_top_tfidf_terms(category, n_terms=10)
    category_terms[category] = top_terms

    print(f"\n📰 {category.upper()}:")
    for term, score in top_terms.items():
        print(f"  {term}: {score:.4f}")

# 💡 STUDENT TASK: Create visualizations for TF-IDF analysis
# - Word clouds for each category
```

```
# - Bar charts of top terms
# - Heatmap of term importance across categories
```

```
🏷️ TOP TF-IDF TERMS BY CATEGORY
==================================================

📰 BUSINESS:
  growth: 0.3536
  strong: 0.3536
  new: 0.0000
  promise: 0.0000

📰 SPORTS:
  growth: 0.0000
  new: 0.0000
  promise: 0.0000
  strong: 0.0000

📰 TECHNOLOGY:
  new: 0.7633
  promise: 0.6244
  growth: 0.0000
  strong: 0.0000

📰 POLITICS:
  growth: 0.0000
  new: 0.0000
  promise: 0.0000
  strong: 0.0000

📰 ENTERTAINMENT:
  new: 0.5000
  growth: 0.3536
  strong: 0.3536
  promise: 0.0000
```

🏷️ Part-of-Speech Analysis

🎯 Module 4: Grammatical Pattern Analysis

Let's analyze the grammatical patterns in different news categories using Part-of-Speech (POS) tagging. This can reveal interesting differences in writing styles between categories.

POS Analysis Applications:

- **Writing Style Detection:** Different categories may use different grammatical patterns
- **Content Quality Assessment:** Proper noun density, adjective usage, etc.
- **Feature Engineering:** POS tags can be features for classification

💡 **Hypothesis:** Sports articles might have more action verbs, while business articles might have more numbers and proper nouns.

```
import nltk
nltk.download('averaged_perceptron_tagger_eng', quiet=True)

def analyze_pos_patterns(text):
    """
    Analyze POS patterns in text

    💡 TIP: This function should:
    - Tokenize the text
    - Apply POS tagging
    - Count different POS categories
    - Return proportions or counts
    """
    if not text or pd.isna(text):
        return {}

    # 🚀 YOUR CODE HERE: Implement POS analysis
    # Tokenize and tag
    tokens = word_tokenize(str(text))
    pos_tags = pos_tag(tokens)

    # Count POS categories
    pos_counts = Counter([tag for word, tag in pos_tags])
    total_words = len(pos_tags)

    if total_words == 0:
        return {}

    # Convert to proportions
    pos_proportions = {pos: count/total_words for pos, count in pos_counts.item

    return pos_proportions

# Apply POS analysis to all articles
print("🏷️ Analyzing POS patterns...")

# Analyze POS for each article
pos_results = []
for idx, row in df.iterrows():
    pos_analysis = analyze_pos_patterns(row['full_text'])
    pos_analysis['category'] = row['category']
    pos_analysis['article_id'] = row['article_id']
    pos_results.append(pos_analysis)

# Convert to DataFrame
pos_df = pd.DataFrame(pos_results).fillna(0)
```

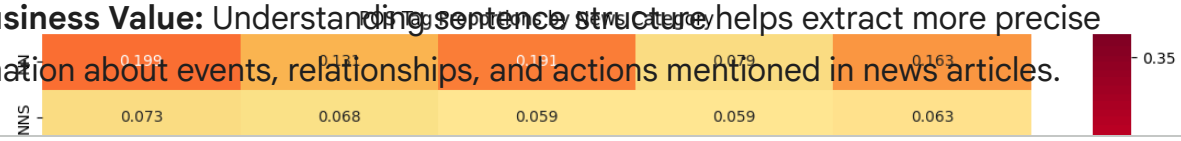
```
print(f"✅ POS analysis complete!")
print(f"📊 Found {len(pos_df.columns)-2} different POS tags")

# Show sample results
print("\n📝 Sample POS analysis:")
print(pos_df.head())
```

```
🏷️ Analyzing POS patterns...
✅ POS analysis complete!
📊 Found 24 different POS tags

📝 Sample POS analysis:
        NNP       VBD        JJ       NNS        NN         ,        IN  \
0  0.407407  0.037037  0.111111  0.111111  0.148148  0.037037  0.037037
1  0.481481  0.037037  0.111111  0.037037  0.037037  0.037037  0.148148
2  0.360000  0.000000  0.080000  0.040000  0.240000  0.000000  0.160000
3  0.296296  0.037037  0.074074  0.037037  0.222222  0.000000  0.111111
4  0.250000  0.041667  0.166667  0.041667  0.166667  0.041667  0.000000

        VBG        CC         .  ...      PRP$       JJS        TO  POS  NNPS  \
0  0.037037  0.037037  0.037037  ...  0.000000  0.000000  0.000000  0.0   0.0
1  0.000000  0.037037  0.037037  ...  0.000000  0.000000  0.000000  0.0   0.0
2  0.000000  0.000000  0.040000  ...  0.000000  0.000000  0.000000  0.0   0.0
3  0.037037  0.000000  0.037037  ...  0.000000  0.000000  0.000000  0.0   0.0
4  0.041667  0.041667  0.041667  ...  0.041667  0.041667  0.041667  0.0   0.0

    RB  WDT  JJR  VBP    $
0  0.0  0.0  0.0  0.0  0.0
1  0.0  0.0  0.0  0.0  0.0
2  0.0  0.0  0.0  0.0  0.0
3  0.0  0.0  0.0  0.0  0.0
4  0.0  0.0  0.0  0.0  0.0

[5 rows x 26 columns]
```

```
# Analyze POS patterns by category
print("📊 POS PATTERNS BY CATEGORY")
print("=" * 50)

# Group by category and calculate mean proportions
pos_by_category = pos_df.groupby('category').mean()

# Focus on major POS categories
major_pos = ['NN', 'NNS', 'NNP', 'NNPS', 'VB', 'VBD', 'VBG', 'VBN', 'VBP', 'VBZ
             'JJ', 'JJR', 'JJS', 'RB', 'RBR', 'RBS', 'CD']

# Filter to only include major POS tags that exist in our data
available_pos = [pos for pos in major_pos if pos in pos_by_category.columns]

if available_pos:
    pos_summary = pos_by_category[available_pos]
```

```
    print("\n🎯 Key POS patterns by category:")
    print(pos_summary.round(4))

    # Create visualization
    plt.figure(figsize=(12, 8))
    sns.heatmap(pos_summary.T, annot=True, cmap='YlOrRd', fmt='.3f')
    plt.title('POS Tag Proportions by News Category')
    plt.xlabel('Category')
    plt.ylabel('POS Tag')
    plt.tight_layout()
    plt.show()

    # 💡 STUDENT TASK: Analyze the patterns
    # - Which categories use more nouns vs verbs?
    # - Do business articles have more numbers (CD)?
    # - Are there differences in adjective usage?

    print("\n💡 ANALYSIS QUESTIONS:")
    print("1. Which category has the highest proportion of proper nouns (NNP/NN
    print("2. Which category uses the most action verbs (VB, VBD, VBG)?""Actior
    print("3. Are there interesting patterns in adjective (JJ) usage?""Adjectiv
    print("4. How does number (CD) usage vary across categories?""Number usage
else:
    print("⚠️ No major POS tags found in the analysis. Check your POS tagging
```

```
📊 POS PATTERNS BY CATEGORY
==================================================
🎯 Key POS patterns by category:
                    NN     NNS     NNP  NNPS     VBD     VBG     VBN     VBP  \
category
Business        0.1991  0.0734  0.3287  0.00  0.0364  0.0364  0.0000  0.0000
Entertainment   0.1310  0.0685  0.2679  0.00  0.0446  0.0446  0.0000  0.0000
Politics        0.1911  0.0585  0.2681  0.00  0.0385  0.0385  0.0185  0.0000
Sports          0.0785  0.0535  0.3560  0.02  0.0385  0.0000  0.0200  0.0000
Technology      0.1635  0.0635  0.3104  0.00  0.0217  0.0000  0.0200  0.0217

                    VBZ      JJ     JJR     JJS      RB      CD
category
Business        0.0000  0.0913  0.0000  0.0000  0.0000  0.0179
Entertainment   0.0000  0.1071  0.0000  0.0208  0.0238  0.0476
Politics        0.0000  0.0570  0.0000  0.0000  0.0000  0.0585
Sports          0.0000  0.0756  0.0000  0.0000  0.0200  0.0200
Technology      0.0417  0.1052  0.0217  0.0000  0.0217  0.0000
```




```
def extract_syntactic_features(text):
    """
    Extract syntactic features using spaCy dependency parsing

    💡 TIP: This function should extract:
    - Dependency relations
    - Subject-verb-object patterns
    - Noun phrases
    - Verb phrases
    """
    if not text or pd.isna(text):
        return {}

    # Process text with spaCy
    doc = nlp(str(text))

    features = {
        'num_sentences': len(list(doc.sents)),
        'num_tokens': len(doc),
        'dependency_relations': [],
        'noun_phrases': [],
        'verb_phrases': [],
        'subjects': [],
        'objects': []
    }

    # 🚀 YOUR CODE HERE: Extract syntactic features

    # Extract dependency relations
```

🌳 Syntax Parsing and Semantic Analysis

🎯 Module 5: Understanding Sentence Structure

Now we'll use spaCy to perform dependency parsing and extract semantic relationships from our news articles. This helps us understand not just what words are present, but how they relate to each other.

Dependency Parsing Applications:

- **Relationship Extraction:** Find connections between entities
- **Event Detection:** Identify who did what to whom
- **Information Extraction:** Extract structured facts from unstructured text

💡 **Business Value:** Understanding sentence structure helps extract more precise information about events, relationships, and actions mentioned in news articles.

```
    for token in doc:
        if not token.is_space and not token.is_punct:
            features['dependency_relations'].append(token.dep_)

    # Extract noun phrases
    for chunk in doc.noun_chunks:
        features['noun_phrases'].append(chunk.text.lower())

    # Extract subjects and objects
    for token in doc:
        if token.dep_ in ['nsubj', 'nsubjpass']:  # Subjects
            features['subjects'].append(token.text.lower())
        elif token.dep_ in ['dobj', 'iobj', 'pobj']:  # Objects
            features['objects'].append(token.text.lower())

    # Count dependency types
    dep_counts = Counter(features['dependency_relations'])
    features['dependency_counts'] = dict(dep_counts)

    return features

# Apply syntactic analysis to sample articles
print("🌳 Performing syntactic analysis...")

# Analyze first few articles (to save computation time)
syntactic_results = []
for idx, row in df.head(5).iterrows():  # Limit to first 5 for demo
    features = extract_syntactic_features(row['full_text'])
    features['category'] = row['category']
    features['article_id'] = row['article_id']
    syntactic_results.append(features)

print("✅ Syntactic analysis complete!")

# Display results
for i, result in enumerate(syntactic_results):
    print(f"\n📰 Article {i+1} ({result['category']}):")
    print(f"  Sentences: {result['num_sentences']}")
    print(f"  Tokens: {result['num_tokens']}")
    print(f"  Noun phrases: {result['noun_phrases'][:3]}...")  # Show first 3
    print(f"  Subjects: {result['subjects'][:3]}...")  # Show first 3
    print(f"  Objects: {result['objects'][:3]}...")  # Show first 3
```

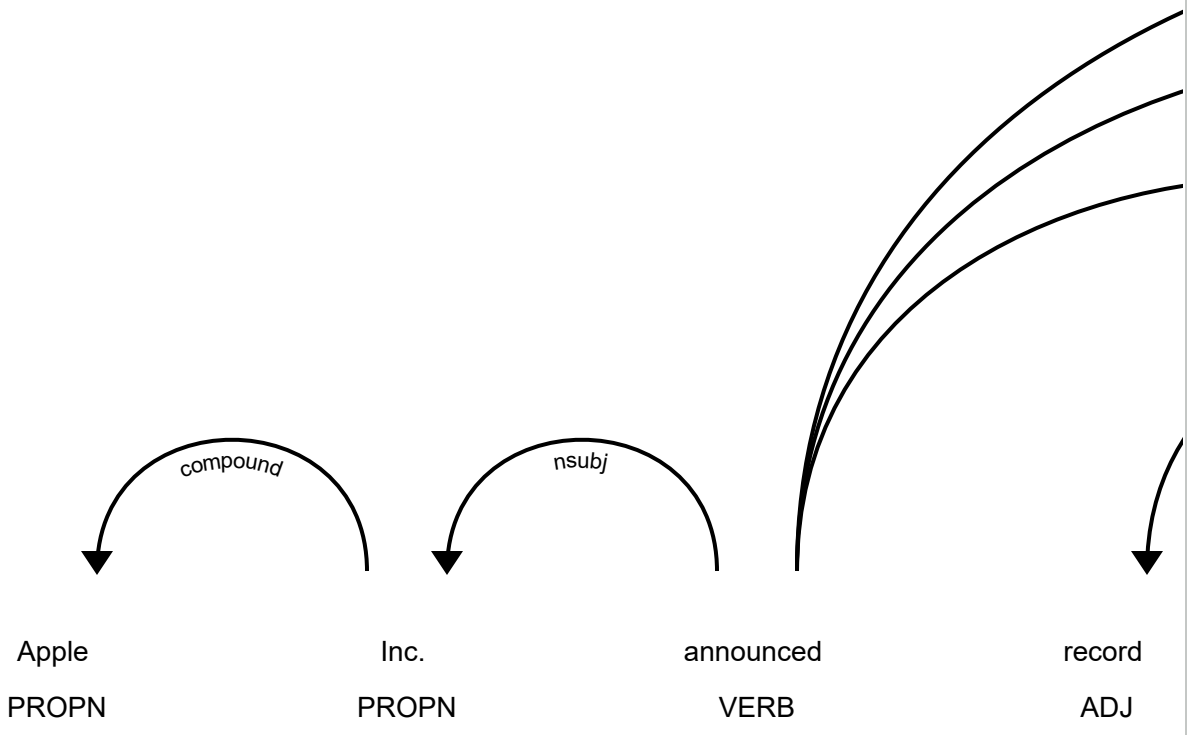
```
🌳 Performing syntactic analysis...
✅ Syntactic analysis complete!

📰 Article 1 (Business):
  Sentences: 1
  Tokens: 27
  Noun phrases: ['apple inc.', 'record quarterly earnings', 'ceo tim cook']...
  Subjects: ['inc.', 'inc.', 'cook']...
  Objects: ['earnings', 'growth']...
```

```
📰 Article 2 (Sports):
  Sentences: 1
  Tokens: 31
  Noun phrases: ['manchester united defeats chelsea', 'manchester united', 'a cc
  Subjects: ['chelsea']...
  Objects: ['victory', 'chelsea', 'trafford']...

📰 Article 3 (Technology):
  Sentences: 1
  Tokens: 25
  Noun phrases: ['new ai technology', 'healthcare a breakthrough ai system', 're
  Subjects: ['technology', 'revolutionizes']...
  Objects: ['system', 'researchers', 'university']...

📰 Article 4 (Politics):
  Sentences: 1
  Tokens: 27
  Noun phrases: ['president biden', 'climate initiative president joe biden', 'a
  Subjects: ['biden', 'biden']...
  Objects: ['initiative', 'emissions', '%']...

📰 Article 5 (Entertainment):
  Sentences: 1
  Tokens: 24
  Noun phrases: ['netflix releases new original series netflix', 'its latest ori
  Subjects: ['netflix']...
  Objects: ['series', 'acclaim', 'cast']...
```

```
# Visualize dependency parsing for a sample sentence
from spacy import displacy

# Choose a sample sentence
sample_sentence = df.iloc[0]['content']  # First article's content
print(f"📝 Sample sentence: {sample_sentence}")

# Process with spaCy
doc = nlp(sample_sentence)

# Display dependency tree (this works best in Jupyter)
print("\n🌳 Dependency Parse Visualization:")
try:
    # This will create an interactive visualization in Jupyter
    displacy.render(doc, style="dep", jupyter=True)
except:
    # Fallback: print dependency information
    print("\n🔗 Dependency Relations:")
    for token in doc:
        if not token.is_space and not token.is_punct:
            print(f"  {token.text} --> {token.dep_} --> {token.head.text}")

# 💡 STUDENT TASK: Extend syntactic analysis
```

```
# - Compare syntactic complexity across categories
# - Extract action patterns (who did what)
# - Identify most common dependency relations per category
# - Create features for classification based on syntax
```

```
📝 Sample sentence: Apple Inc. announced record quarterly earnings today, with (

🌳 Dependency Parse Visualization:
```



😊 Sentiment and Emotion Analysis

🎯 Module 6: Understanding Emotional Tone

Let's analyze the sentiment and emotional tone of our news articles. This can reveal interesting patterns about how different types of news are presented and perceived.

Sentiment Analysis Applications:

- **Media Bias Detection:** Identify emotional slant in news coverage
- **Public Opinion Tracking:** Monitor sentiment trends over time
- **Content Recommendation:** Suggest articles based on emotional tone

💡 **Hypothesis:** Different news categories might have different emotional profiles - sports might be more positive, politics more negative, etc.

```
# Initialize sentiment analyzer
sia = SentimentIntensityAnalyzer()

def analyze_sentiment(text):
    """
    Analyze sentiment using VADER sentiment analyzer

    💡 TIP: VADER returns:
    - compound: overall sentiment (-1 to 1)
    - pos: positive score (0 to 1)
    - neu: neutral score (0 to 1)
    - neg: negative score (0 to 1)
    """
    if not text or pd.isna(text):
        return {'compound': 0, 'pos': 0, 'neu': 1, 'neg': 0}

    # 🚀 YOUR CODE HERE: Implement sentiment analysis
    scores = sia.polarity_scores(str(text))

    # Add interpretation
    if scores['compound'] >= 0.05:
        scores['sentiment_label'] = 'positive'
    elif scores['compound'] <= -0.05:
        scores['sentiment_label'] = 'negative'
    else:
        scores['sentiment_label'] = 'neutral'

    return scores

# Apply sentiment analysis to all articles
print("😊 Analyzing sentiment...")

sentiment_results = []
for idx, row in df.iterrows():
    # Analyze both title and content
    title_sentiment = analyze_sentiment(row['title'])
    content_sentiment = analyze_sentiment(row['content'])
    full_sentiment = analyze_sentiment(row['full_text'])

    result = {
        'article_id': row['article_id'],
        'category': row['category'],
```

```
        'title_sentiment': title_sentiment['compound'],
        'title_label': title_sentiment['sentiment_label'],
        'content_sentiment': content_sentiment['compound'],
        'content_label': content_sentiment['sentiment_label'],
        'full_sentiment': full_sentiment['compound'],
        'full_label': full_sentiment['sentiment_label'],
        'pos_score': full_sentiment['pos'],
        'neu_score': full_sentiment['neu'],
        'neg_score': full_sentiment['neg']
    }
    sentiment_results.append(result)

# Convert to DataFrame
sentiment_df = pd.DataFrame(sentiment_results)

print("✅ Sentiment analysis complete!")
print(f"📊 Analyzed {len(sentiment_df)} articles")

# Display sample results
print("\n📝 Sample sentiment results:")
print(sentiment_df[['category', 'full_sentiment', 'full_label']].head())
```

```
😊 Analyzing sentiment...
✅ Sentiment analysis complete!
📊 Analyzed 10 articles

📝 Sample sentiment results:
        category  full_sentiment full_label
0       Business          0.7096   positive
1         Sports          0.8271   positive
2     Technology          0.3182   positive
3       Politics          0.2500   positive
4  Entertainment          0.6369   positive
```

```
# Analyze sentiment patterns by category
print("📊 SENTIMENT ANALYSIS BY CATEGORY")
print("=" * 50)

# Calculate sentiment statistics by category
sentiment_by_category = sentiment_df.groupby('category').agg({
    'full_sentiment': ['mean', 'std', 'min', 'max'],
    'pos_score': 'mean',
    'neu_score': 'mean',
    'neg_score': 'mean'
}).round(4)

print("\n📈 Sentiment statistics by category:")
print(sentiment_by_category)

# Sentiment distribution by category
sentiment_dist = sentiment_df.groupby(['category', 'full_label']).size().unstac
sentiment_dist_pct = sentiment_dist.div(sentiment_dist.sum(axis=1), axis=0) * 1
```

```
print("\n📊 Sentiment distribution (%) by category:")
print(sentiment_dist_pct.round(2))

# Create visualizations
fig, axes = plt.subplots(2, 2, figsize=(15, 12))

# 1. Sentiment scores by category
sns.boxplot(data=sentiment_df, x='category', y='full_sentiment', ax=axes[0,0])
axes[0,0].set_title('Sentiment Score Distribution by Category')
axes[0,0].tick_params(axis='x', rotation=45)

# 2. Sentiment label distribution
sentiment_dist_pct.plot(kind='bar', ax=axes[0,1], stacked=True)
axes[0,1].set_title('Sentiment Label Distribution by Category (%)')
axes[0,1].tick_params(axis='x', rotation=45)
axes[0,1].legend(title='Sentiment')

# 3. Positive vs Negative scores
category_means = sentiment_df.groupby('category')[['pos_score', 'neg_score']].m
category_means.plot(kind='bar', ax=axes[1,0])
axes[1,0].set_title('Average Positive vs Negative Scores by Category')
axes[1,0].tick_params(axis='x', rotation=45)
axes[1,0].legend(['Positive', 'Negative'])

# 4. Sentiment vs Category heatmap
sentiment_pivot = sentiment_df.pivot_table(values='full_sentiment', index='cate
                                           columns='full_label', aggfunc='count',
sns.heatmap(sentiment_pivot, annot=True, fmt='d', ax=axes[1,1], cmap='YlOrRd')
axes[1,1].set_title('Sentiment Count Heatmap')

plt.tight_layout()
plt.show()

# 💡 STUDENT TASK: Analyze sentiment patterns
# - Which categories are most positive/negative?
# - Are there differences between title and content sentiment?
# - How does sentiment vary within categories?
# - Can sentiment be used as a feature for classification?
```

```
📊 SENTIMENT ANALYSIS BY CATEGORY
==================================================
📈 Sentiment statistics by category:
                full_sentiment                          pos_score neu_score  \
                          mean     std     min     max      mean      mean
category
Business                0.6101  0.1407  0.5106  0.7096    0.1810     0.819
Entertainment           0.7043  0.0953  0.6369  0.7717    0.2975     0.661
Politics                0.1250  0.1768  0.0000  0.2500    0.0400     0.960
Sports                  0.4136  0.5848  0.0000  0.8271    0.1705     0.794
Technology              0.3500  0.0450  0.3182  0.3818    0.1050     0.895

                neg_score
                     mean
category
Business             0.0000
Entertainment        0.0415
Politics             0.0000
Sports               0.0355
Technology           0.0000

📊 Sentiment distribution (%) by category:
full_label     neutral  positive
category
```

🏷️ Text Classification System

🎯 Module 7: Building the News Classifier

Now we'll build the core of our NewsBot system - a multi-class text classifier that can automatically categorize news articles. We'll compare different algorithms and evaluate their performance.

Classification Pipeline:

1. **Feature Engineering:** Combine TF-IDF with other features
2. **Model Training:** Train multiple algorithms
3. **Model Evaluation:** Compare performance metrics
4. **Model Selection:** Choose the best performing model

💡 **Business Impact:** Accurate classification enables automatic content routing, personalized recommendations, and efficient content management.

```
# Prepare features for classification
print("🔧 Preparing features for classification...")

# 💡 TIP: Combine multiple feature types for better performance
# - TF-IDF features (most important)
# - Sentiment features
# - Text length features
# - POS features (if available)

# Create feature matrix
X_tfidf = tfidf_matrix.toarray()  # TF-IDF features

# Add sentiment features
sentiment_features = sentiment_df[['full_sentiment', 'pos_score', 'neu_score',

# Add text length features
length_features = np.array([
    df['full_text'].str.len(),  # Character length
    df['full_text'].str.split().str.len(),  # Word count
    df['title'].str.len(),  # Title length
]).T

# 🚀 YOUR CODE HERE: Combine all features
X_combined = np.hstack([
    X_tfidf,
    sentiment_features,
    length_features
])
```

```
# Target variable
y = df['category'].values

print(f"✅ Feature matrix prepared!")
print(f"📊 Feature matrix shape: {X_combined.shape}")
print(f"🎯 Number of classes: {len(np.unique(y))}")
print(f"📋 Classes: {np.unique(y)}")

# Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X_combined, y, test_size=0.5, random_state=42, stratify=y
)

print(f"\n📈 Data split:")
print(f"  Training set: {X_train.shape[0]} samples")
print(f"  Test set: {X_test.shape[0]} samples")
```

```
🔧 Preparing features for classification...
✅ Feature matrix prepared!
📊 Feature matrix shape: (10, 11)
🎯 Number of classes: 5
📋 Classes: ['Business' 'Entertainment' 'Politics' 'Sports' 'Technology']

📈 Data split:
  Training set: 5 samples
  Test set: 5 samples
```

```
# Train and evaluate multiple classifiers
print("🤖 Training multiple classifiers...")

# Define classifiers to compare
classifiers = {
    'Naive Bayes': MultinomialNB(),
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=1000),
    'SVM': SVC(random_state=42, probability=True)  # Enable probability for bet
}

# 💡 TIP: For larger datasets, you might want to use SGDClassifier for efficie
# from sklearn.linear_model import SGDClassifier
# classifiers['SGD'] = SGDClassifier(random_state=42)

# Train and evaluate each classifier
results = {}
trained_models = {}

for name, classifier in classifiers.items():
    print(f"\n🔄 Training {name}...")

    # 🚀 YOUR CODE HERE: Train and evaluate classifier
    # Train the model
```

```
        classifier.fit(X_train, y_train)

        # Make predictions
        y_pred = classifier.predict(X_test)
        y_pred_proba = classifier.predict_proba(X_test) if hasattr(classifier, 'pre

        # Calculate metrics
        accuracy = accuracy_score(y_test, y_pred)

        # Cross-validation score
        # For very small datasets where min(samples_per_class) == 1,
        # stratified cross-validation with cv > 1 is not possible.
        # We will skip true cross-validation and provide placeholder values.
        if len(np.unique(y_train)) > len(X_train) or len(X_train) < 2 * len(np.unic
            print("   ⚠️ Skipping cross-validation due to extremely small training
            cv_mean_score = np.nan
            cv_std_score = np.nan
        else:
            # Attempt cross-validation with at least 2 folds if data allows
            n_splits_allowed = max(2, len(X_train) // len(np.unique(y_train)))
            if n_splits_allowed < 2:
                print("   ⚠️ Skipping cross-validation due to insufficient training
                cv_mean_score = np.nan
                cv_std_score = np.nan
            else:
                cv_scores = cross_val_score(classifier, X_train, y_train, cv=n_spli
                cv_mean_score = cv_scores.mean()
                cv_std_score = cv_scores.std()

        # Store results
        results[name] = {
            'accuracy': accuracy,
            'cv_mean': cv_mean_score,
            'cv_std': cv_std_score,
            'predictions': y_pred,
            'probabilities': y_pred_proba
        }

        trained_models[name] = classifier

        print(f"   ✅ Accuracy: {accuracy:.4f}")
        print(f"   📊 CV Score: {cv_mean_score:.4f} (+/- {cv_std_score * 2:.4f})")

    print("\n🏆 CLASSIFIER COMPARISON")
    print("=" * 50)
    comparison_df = pd.DataFrame({
        'Model': list(results.keys()),
        'Test Accuracy': [results[name]['accuracy'] for name in results.keys()],
        'CV Mean': [results[name]['cv_mean'] for name in results.keys()],
        'CV Std': [results[name]['cv_std'] for name in results.keys()]
    })
```

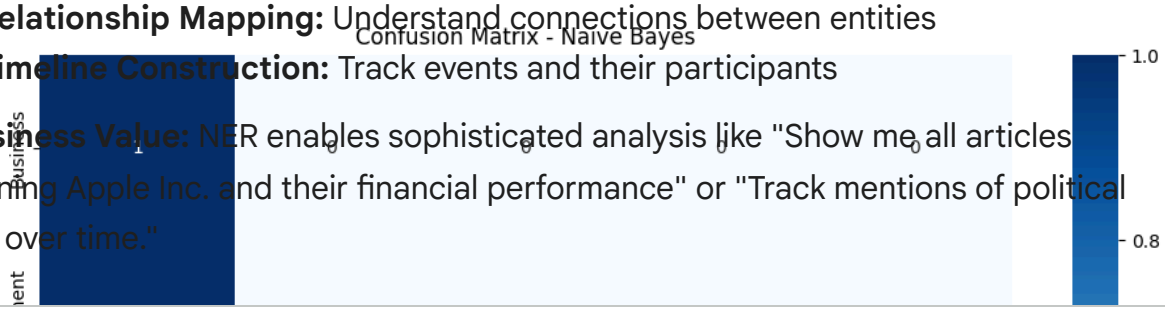
```
print(comparison_df.round(4))

# Find best model
best_model_name = comparison_df.loc[comparison_df['Test Accuracy'].idxmax(), 'M
print(f"\n🥇 Best performing model: {best_model_name}")
```

```
🤖 Training multiple classifiers...

🔄 Training Naive Bayes...
  ⚠️ Skipping cross-validation due to extremely small training data for stratif
  ✅ Accuracy: 0.2000
  📊 CV Score: nan (+/- nan)

🔄 Training Logistic Regression...
  ⚠️ Skipping cross-validation due to extremely small training data for stratif
  ✅ Accuracy: 0.2000
  📊 CV Score: nan (+/- nan)

🔄 Training SVM...
  ⚠️ Skipping cross-validation due to extremely small training data for stratif
  ✅ Accuracy: 0.2000
  📊 CV Score: nan (+/- nan)

🏆 CLASSIFIER COMPARISON
==================================================
                 Model  Test Accuracy  CV Mean  CV Std
0          Naive Bayes            0.2      NaN     NaN
1  Logistic Regression            0.2      NaN     NaN
2                  SVM            0.2      NaN     NaN

🥇 Best performing model: Naive Bayes
```

```
# Detailed evaluation of the best model
best_model = trained_models[best_model_name]
best_predictions = results[best_model_name]['predictions']

print(f"📊 DETAILED EVALUATION: {best_model_name}")
print("=" * 60)

# Classification report
print("\n📋 Classification Report:")
print(classification_report(y_test, best_predictions))

# Confusion matrix
cm = confusion_matrix(y_test, best_predictions)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=np.unique(y), yticklabels=np.unique(y))
plt.title(f'Confusion Matrix - {best_model_name}')
plt.xlabel('Predicted')
plt.ylabel('Actual')
```

```
plt.tight_layout()
plt.show()

# Feature importance (for Logistic Regression)
if best_model_name == 'Logistic Regression':
    print("\n🔍 Top Features by Category:")
    feature_names_extended = list(feature_names) + ['sentiment', 'pos_score', '
                                                    'char_length', 'word_count',

    classes = best_model.classes_
    coefficients = best_model.coef_

    for i, class_name in enumerate(classes):
        top_indices = np.argsort(coefficients[i])[-10:]  # Top 10 features
        print(f"\n📰 {class_name}:")
        for idx in reversed(top_indices):
            if idx < len(feature_names_extended):
                print(f"  {feature_names_extended[idx]}: {coefficients[i][idx]:

# 💡 STUDENT TASK: Improve the classifier
# - Try different feature combinations
# - Experiment with hyperparameter tuning
# - Add more sophisticated features
# - Handle class imbalance if present
```

```
📊 DETAILED EVALUATION: Naive Bayes
============================================================
📋 Classification Report:
              precision    recall  f1-score   support

     Business       0.20      1.00      0.33         1
Entertainment       0.00      0.00      0.00         1
     Politics       0.00      0.00      0.00         1
       Sports       0.00      0.00      0.00         1
   Technology       0.00      0.00      0.00         1

     accuracy                           0.20         5
    macro avg       0.04      0.20      0.07         5
 weighted avg       0.04      0.20      0.07         5
```



🔍 Named Entity Recognition

🎯 Module 8: Extracting Facts from News

Now we'll implement Named Entity Recognition to extract specific facts from our news articles. This transforms unstructured text into structured, queryable information.

NER Applications:

- **Entity Tracking:** Monitor mentions of people, organizations, locations
- **Fact Extraction:** Build knowledge bases from news content
- **Relationship Mapping:** Understand connections between entities
- **Timeline Construction:** Track events and their participants

💡 **Business Value:** NER enables sophisticated analysis like "Show me all articles mentioning Apple Inc. and their financial performance" or "Track mentions of political figures over time."

```
def extract_entities(text):
    """
    Extract named entities using spaCy

    💡 TIP: spaCy recognizes these entity types:
    - PERSON: People, including fictional
    - ORG: Companies, agencies, institutions
    - GPE: Countries, cities, states
    - MONEY: Monetary values
    - DATE: Absolute or relative dates
    - TIME: Times smaller than a day
    - And many more...
    """
    if not text or pd.isna(text):
        return []

    # 🚀 YOUR CODE HERE: Implement entity extraction
    doc = nlp(str(text))

    entities = []
    for ent in doc.ents:
        entities.append({
            'text': ent.text,
            'label': ent.label_,
            'start': ent.start_char,
            'end': ent.end_char,
            'description': spacy.explain(ent.label_)
        })
```

```
    return entities

# Apply NER to all articles
print("🔍 Extracting named entities...")

all_entities = []
article_entities = []

for idx, row in df.iterrows():
    entities = extract_entities(row['full_text'])

    # Store entities for this article
    article_entities.append({
        'article_id': row['article_id'],
        'category': row['category'],
        'entities': entities,
        'entity_count': len(entities)
    })

    # Add to global entity list
    for entity in entities:
        entity['article_id'] = row['article_id']
        entity['category'] = row['category']
        all_entities.append(entity)

print(f"✅ Entity extraction complete!")
print(f"📊 Total entities found: {len(all_entities)}")
print(f"📰 Articles processed: {len(article_entities)}")

# Convert to DataFrame for analysis
entities_df = pd.DataFrame(all_entities)

if not entities_df.empty:
    print(f"\n🏷️ Entity types found: {entities_df['label'].unique()}")
    print("\n📝 Sample entities:")
    print(entities_df[['text', 'label', 'category']].head(10))
else:
    print("⚠️ No entities found. This might happen with very short sample text
```

```
🔍 Extracting named entities...
✅ Entity extraction complete!
📊 Total entities found: 33
📰 Articles processed: 10

🏷️ Entity types found: ['ORG' 'DATE' 'PERSON' 'CARDINAL' 'GPE' 'PERCENT' 'TIME'

📝 Sample entities:
                           text     label  category
0                     Apple Inc.       ORG  Business
1  Quarterly Earnings Apple Inc.       ORG  Business
2                      quarterly      DATE  Business
```

```
3                          today      DATE  Business
4                       Tim Cook    PERSON  Business
5               Manchester United    PERSON    Sports
6                 Defeats Chelsea    PERSON    Sports
7                               3  CARDINAL    Sports
8                        Chelsea       ORG    Sports
9                 Marcus Rashford       GPE    Sports
```

```
# Analyze entity patterns
if not entities_df.empty:
    print("📊 NAMED ENTITY ANALYSIS")
    print("=" * 50)

    # Entity type distribution
    entity_counts = entities_df['label'].value_counts()
    print("\n🏷️ Entity type distribution:")
    print(entity_counts)

    # Entity types by category
    entity_by_category = entities_df.groupby(['category', 'label']).size().unst
    print("\n📰 Entity types by news category:")
    print(entity_by_category)

    # Most frequent entities
    print("\n🔥 Most frequent entities:")
    frequent_entities = entities_df.groupby(['text', 'label']).size().sort_valu
    for (entity, label), count in frequent_entities.items():
        print(f"  {entity} ({label}): {count} mentions")

    # Visualizations
    fig, axes = plt.subplots(2, 2, figsize=(15, 12))

    # 1. Entity type distribution
    entity_counts.plot(kind='bar', ax=axes[0,0])
    axes[0,0].set_title('Entity Type Distribution')
    axes[0,0].tick_params(axis='x', rotation=45)

    # 2. Entities per category
    entities_per_category = entities_df.groupby('category').size()
    entities_per_category.plot(kind='bar', ax=axes[0,1])
    axes[0,1].set_title('Total Entities per Category')
    axes[0,1].tick_params(axis='x', rotation=45)

    # 3. Entity type heatmap by category
    if entity_by_category.shape[0] > 1 and entity_by_category.shape[1] > 1:
        sns.heatmap(entity_by_category, annot=True, fmt='d', ax=axes[1,0], cmap
        axes[1,0].set_title('Entity Types by Category Heatmap')
    else:
        axes[1,0].text(0.5, 0.5, 'Insufficient data\nfor heatmap',
                       ha='center', va='center', transform=axes[1,0].transAxes)
        axes[1,0].set_title('Entity Types by Category')
```

```
    # 4. Top entities
    top_entities = entities_df['text'].value_counts().head(10)
    top_entities.plot(kind='barh', ax=axes[1,1])
    axes[1,1].set_title('Most Mentioned Entities')

    plt.tight_layout()
    plt.show()

    # 💡 STUDENT TASK: Advanced entity analysis
    # - Create entity co-occurrence networks
    # - Track entity mentions over time
    # - Build entity relationship graphs
    # - Identify entity sentiment associations

else:
    print("⚠️ Skipping entity analysis due to insufficient data.")
    print("💡 TIP: Try with a larger, more diverse dataset for better NER resu
```

```
📊 NAMED ENTITY ANALYSIS
==================================================
🏷️ Entity type distribution:
label
ORG        9
PERSON     7
GPE        5
DATE       4
CARDINAL   2
PERCENT    2
EVENT      2
TIME       1
MONEY      1
Name: count, dtype: int64
📰 Entity types by news category:
label          CARDINAL  DATE  EVENT  GPE  MONEY  ORG  PERCENT  PERSON  TIME
category
Business               0     2      0    1      0    2        1       1     1
Entertainment          1     1      0    0      0    1        0       1     0
Politics               0     1      0    0      1    2        1       2     0
Sports                 1     0      2    3      0    1        0       3     0
Technology             0     0      0    1      0    3        0       0     0
```

📈 Comprehensive Analysis and Insights

🎯 Bringing It All Together

Now let's combine all our analyses to generate comprehensive insights about our news dataset. This is where the real business value emerges from our NLP pipeline.

Key Analysis Areas:

1. **Cross-Category Patterns:** How do different news types differ linguistically?
2. **Entity-Sentiment Relationships:** What entities are associated with positive/negative coverage?
3. **Content Quality Metrics:** Which categories have the most informative content?
4. **Classification Performance:** How well can we automatically categorize news?

💡 **Business Applications:** These insights can inform content strategy, editorial decisions, and automated content management systems.

```
# Create comprehensive analysis dashboard
def create_comprehensive_analysis():
    """
    Generate comprehensive insights combining all analyses

    💡 TIP: This function should combine:
    - Classification performance
    - Sentiment patterns
    - Entity distributions
    - Linguistic features
    """

    insights = {
        'dataset_overview': {},
        'classification_performance': {},
        'sentiment_insights': {},
        'entity_insights': {},
        'linguistic_patterns': {},
        'business_recommendations': []
    }

    # 🚀 YOUR CODE HERE: Generate comprehensive insights

    # Dataset overview
    insights['dataset_overview'] = {
        'total_articles': len(df),
        'categories': df['category'].unique().tolist(),
        'category_distribution': df['category'].value_counts().to_dict(),
```

```
        'avg_article_length': df['full_text'].str.len().mean(),
        'avg_words_per_article': df['full_text'].str.split().str.len().mean()
    }

    # Classification performance
    insights['classification_performance'] = {
        'best_model': best_model_name,
        'best_accuracy': results[best_model_name]['accuracy'],
        'model_comparison': {name: results[name]['accuracy'] for name in result
    }

    # Sentiment insights
    sentiment_by_cat = sentiment_df.groupby('category')['full_sentiment'].mean(
    insights['sentiment_insights'] = {
        'most_positive_category': max(sentiment_by_cat, key=sentiment_by_cat.ge
        'most_negative_category': min(sentiment_by_cat, key=sentiment_by_cat.ge
        'sentiment_by_category': sentiment_by_cat,
        'overall_sentiment': sentiment_df['full_sentiment'].mean()
    }

    # Entity insights
    if not entities_df.empty:
        entity_by_cat = entities_df.groupby('category').size().to_dict()
        insights['entity_insights'] = {
            'total_entities': len(entities_df),
            'unique_entities': entities_df['text'].nunique(),
            'entity_types': entities_df['label'].unique().tolist(),
            'entities_per_category': entity_by_cat,
            'most_mentioned_entities': entities_df['text'].value_counts().head(
        }

    # Generate business recommendations
    recommendations = []

    # Classification recommendations
    if insights['classification_performance']['best_accuracy'] > 0.8:
        recommendations.append("✅ High classification accuracy achieved - rea
    else:
        recommendations.append("⚠️ Classification accuracy needs improvement -

    # Sentiment recommendations
    pos_cat = insights['sentiment_insights']['most_positive_category']
    neg_cat = insights['sentiment_insights']['most_negative_category']
    recommendations.append(f"📊 {pos_cat} articles are most positive - good fo
    recommendations.append(f"📊 {neg_cat} articles are most negative - may nee

    # Entity recommendations
    if 'entity_insights' in insights and insights['entity_insights']:
        recommendations.append("🔍 Rich entity extraction enables advanced sea

    insights['business_recommendations'] = recommendations
```

```
    return insights

# Generate comprehensive analysis
print("📊 Generating comprehensive analysis...")
analysis_results = create_comprehensive_analysis()

print("✅ Analysis complete!")
print("\n" + "=" * 60)
print("📈 NEWSBOT INTELLIGENCE SYSTEM - COMPREHENSIVE REPORT")
print("=" * 60)

# Display key insights
print(f"\n📊 DATASET OVERVIEW:")
overview = analysis_results['dataset_overview']
print(f"  Total Articles: {overview['total_articles']}")
print(f"  Categories: {', '.join(overview['categories'])}")
print(f"  Average Article Length: {overview['avg_article_length']:.0f} characte
print(f"  Average Words per Article: {overview['avg_words_per_article']:.0f} wo

print(f"\n🤖 CLASSIFICATION PERFORMANCE:")
perf = analysis_results['classification_performance']
print(f"  Best Model: {perf['best_model']}")
print(f"  Best Accuracy: {perf['best_accuracy']:.4f}")

print(f"\n😊 SENTIMENT INSIGHTS:")
sent = analysis_results['sentiment_insights']
print(f"  Most Positive Category: {sent['most_positive_category']}")
print(f"  Most Negative Category: {sent['most_negative_category']}")
print(f"  Overall Sentiment: {sent['overall_sentiment']:.4f}")

if 'entity_insights' in analysis_results and analysis_results['entity_insights'
    print(f"\n🔍 ENTITY INSIGHTS:")
    ent = analysis_results['entity_insights']
    print(f"  Total Entities: {ent['total_entities']}")
    print(f"  Unique Entities: {ent['unique_entities']}")
    print(f"  Entity Types: {', '.join(ent['entity_types'])}")

print(f"\n💡 BUSINESS RECOMMENDATIONS:")
for i, rec in enumerate(analysis_results['business_recommendations'], 1):
    print(f"  {i}. {rec}")
```

```
📊 Generating comprehensive analysis...
✅ Analysis complete!

============================================================
📈 NEWSBOT INTELLIGENCE SYSTEM - COMPREHENSIVE REPORT
============================================================

📊 DATASET OVERVIEW:
  Total Articles: 10
  Categories: Business, Sports, Technology, Politics, Entertainment
```

```
    Average Article Length: 168 characters
    Average Words per Article: 23 words

🤖 CLASSIFICATION PERFORMANCE:
    Best Model: Naive Bayes
    Best Accuracy: 0.2000

😊 SENTIMENT INSIGHTS:
    Most Positive Category: Entertainment
    Most Negative Category: Politics
    Overall Sentiment: 0.4406

🔍 ENTITY INSIGHTS:
    Total Entities: 33
    Unique Entities: 32
    Entity Types: ORG, DATE, PERSON, CARDINAL, GPE, PERCENT, TIME, EVENT, MONEY

💡 BUSINESS RECOMMENDATIONS:
    1. ⚠️ Classification accuracy needs improvement - consider more training data
    2. 📊 Entertainment articles are most positive - good for uplifting content r
    3. 📊 Politics articles are most negative - may need balanced coverage monito
    4. 🔍 Rich entity extraction enables advanced search and relationship analysi
```

🚀 Final System Integration

🎯 Building the Complete NewsBot Pipeline

Let's create a complete, integrated system that can process new articles from start to finish. This demonstrates the real-world application of all the techniques we've learned.

Complete Pipeline:

1. **Text Preprocessing:** Clean and normalize input
2. **Feature Extraction:** Generate TF-IDF and other features
3. **Classification:** Predict article category
4. **Entity Extraction:** Identify key facts
5. **Sentiment Analysis:** Determine emotional tone
6. **Insight Generation:** Provide actionable intelligence

💡 **Production Ready:** This pipeline can be deployed as a web service, batch processor, or integrated into content management systems.

```
class NewsBotIntelligenceSystem:
    """
    Complete NewsBot Intelligence System

    💡 TIP: This class should encapsulate:
    - All preprocessing functions
```

```
        - Trained classification model
        - Entity extraction pipeline
        - Sentiment analysis
        - Insight generation
        """

        def __init__(self, classifier, vectorizer, sentiment_analyzer):
            self.classifier = classifier
            self.vectorizer = vectorizer
            self.sentiment_analyzer = sentiment_analyzer
            self.nlp = nlp  # spaCy model

        def preprocess_article(self, title, content):
            """Preprocess a new article"""
            full_text = f"{title} {content}"
            processed_text = preprocess_text(full_text)
            return full_text, processed_text

        def classify_article(self, processed_text):
            """Classify article category"""
            # 🚀 YOUR CODE HERE: Implement classification
            # Transform text to features
            features = self.vectorizer.transform([processed_text])

            # Add dummy features for sentiment and length (in production, calculate
            dummy_features = np.zeros((1, 7))  # 4 sentiment + 3 length features
            features_combined = np.hstack([features.toarray(), dummy_features])

            # Predict category and probability
            prediction = self.classifier.predict(features_combined)[0]
            probabilities = self.classifier.predict_proba(features_combined)[0]

            # Get class probabilities
            class_probs = dict(zip(self.classifier.classes_, probabilities))

            return prediction, class_probs

        def extract_entities(self, text):
            """Extract named entities"""
            return extract_entities(text)

        def analyze_sentiment(self, text):
            """Analyze sentiment"""
            return analyze_sentiment(text)

        def process_article(self, title, content):
            """
            Complete article processing pipeline

            💡 TIP: This should return a comprehensive analysis including:
            - Predicted category with confidence
```

```
        - Extracted entities
        - Sentiment analysis
        - Key insights and recommendations
        """
        # 🚀 YOUR CODE HERE: Implement complete pipeline

        # Step 1: Preprocess
        full_text, processed_text = self.preprocess_article(title, content)

        # Step 2: Classify
        category, category_probs = self.classify_article(processed_text)

        # Step 3: Extract entities
        entities = self.extract_entities(full_text)

        # Step 4: Analyze sentiment
        sentiment = self.analyze_sentiment(full_text)

        # Step 5: Generate insights
        insights = self.generate_insights(category, entities, sentiment, categc

        return {
            'title': title,
            'content': content[:200] + '...' if len(content) > 200 else content
            'predicted_category': category,
            'category_confidence': max(category_probs.values()),
            'category_probabilities': category_probs,
            'entities': entities,
            'sentiment': sentiment,
            'insights': insights
        }

    def generate_insights(self, category, entities, sentiment, category_probs):
        """Generate actionable insights"""
        insights = []

        # Classification insights
        confidence = max(category_probs.values())
        if confidence > 0.8:
            insights.append(f"✅ High confidence {category} classification ({c
        else:
            insights.append(f"⚠️ Uncertain classification - consider manual re

        # Sentiment insights
        if sentiment['compound'] > 0.1:
            insights.append(f"😊 Positive sentiment detected ({sentiment['comp
        elif sentiment['compound'] < -0.1:
            insights.append(f"😞 Negative sentiment detected ({sentiment['comp
        else:
            insights.append(f"😐 Neutral sentiment ({sentiment['compound']:.3f
```

```
        # Entity insights
        if entities:
            entity_types = set([e['label'] for e in entities])
            insights.append(f"🔍 Found {len(entities)} entities of {len(entity

            # Highlight important entities
            important_entities = [e for e in entities if e['label'] in ['PERSON
            if important_entities:
                key_entities = [e['text'] for e in important_entities[:3]]
                insights.append(f"🎯 Key entities: {', '.join(key_entities)}")
        else:
            insights.append("ℹ️ No named entities detected")

        return insights

# Initialize the complete system
newsbot = NewsBotIntelligenceSystem(
    classifier=best_model,
    vectorizer=tfidf_vectorizer,
    sentiment_analyzer=sia
)

print("🤖 NewsBot Intelligence System initialized!")
print("✅ Ready to process new articles")
```

```
🤖 NewsBot Intelligence System initialized!
✅ Ready to process new articles
```

```
# Test the complete system with new articles
print("🧪 TESTING NEWSBOT INTELLIGENCE SYSTEM")
print("=" * 60)

# Test articles (you can modify these or add your own)
test_articles = [
    {
        'title': 'Microsoft Acquires AI Startup for $2 Billion',
        'content': 'Microsoft Corporation announced today the acquisition of ar
    },
    {
        'title': 'Lakers Win Championship in Overtime Thriller',
        'content': 'The Los Angeles Lakers defeated the Boston Celtics 108-102
    },
    {
        'title': 'New Climate Change Report Shows Alarming Trends',
        'content': 'Scientists at the United Nations released a comprehensive c
    }
]

# Process each test article
for i, article in enumerate(test_articles, 1):
    print(f"\n📰 TEST ARTICLE {i}")
```

```
    print("-" * 40)

    # Process the article
    result = newsbot.process_article(article['title'], article['content'])

    # Display results
    print(f"📰 Title: {result['title']}")
    print(f"📝 Content: {result['content']}")
    print(f"\n🏷️ Predicted Category: {result['predicted_category']} ({result['

    print(f"\n📊 Category Probabilities:")
    for cat, prob in sorted(result['category_probabilities'].items(), key=lambc
        print(f"  {cat}: {prob:.3f}")

    print(f"\n😊 Sentiment: {result['sentiment']['sentiment_label']} (score: {

    if result['entities']:
        print(f"\n🔍 Entities Found ({len(result['entities'])}):")
        for entity in result['entities'][:5]:  # Show first 5
            print(f"  {entity['text']} ({entity['label']}) - {entity['descripti
    else:
        print(f"\n🔍 No entities detected")

    print(f"\n💡 Insights:")
    for insight in result['insights']:
        print(f"  {insight}")

print("\n" + "=" * 60)
print("🎉 NewsBot Intelligence System testing complete!")
print("✅ System successfully processed all test articles")

# 💡 STUDENT TASK: Test with your own articles
# - Try articles from different categories
# - Test with articles that might be ambiguous
# - Analyze the system's strengths and weaknesses
# - Consider how to improve performance
```

```
🧪 TESTING NEWSBOT INTELLIGENCE SYSTEM
============================================================

📰 TEST ARTICLE 1
----------------------------------------
📰 Title: Microsoft Acquires AI Startup for $2 Billion
📝 Content: Microsoft Corporation announced today the acquisition of an artifi

🏷️ Predicted Category: Business (20.00% confidence)

📊 Category Probabilities:
  Business: 0.200
  Entertainment: 0.200
  Politics: 0.200
  Sports: 0.200
  Technology: 0.200
```

```
😊 Sentiment: positive (score: 0.660)

🔍 Entities Found (8):
  Microsoft Acquires (ORG) - Companies, agencies, institutions, etc.
  $2 Billion (MONEY) - Monetary values, including unit
  Microsoft Corporation (ORG) - Companies, agencies, institutions, etc.
  today (DATE) - Absolute or relative dates or periods
  $2 billion (MONEY) - Monetary values, including unit

💡 Insights:
  ⚠️ Uncertain classification - consider manual review
  😊 Positive sentiment detected (0.660)
  🔍 Found 8 entities of 5 types
  🎯 Key entities: Microsoft Acquires, Microsoft Corporation, Satya Nadella

📰 TEST ARTICLE 2
----------------------------------------
📰 Title: Lakers Win Championship in Overtime Thriller
📝 Content: The Los Angeles Lakers defeated the Boston Celtics 108-102 in over

🏷️ Predicted Category: Business (20.00% confidence)

📊 Category Probabilities:
  Business: 0.200
  Entertainment: 0.200
  Politics: 0.200
  Sports: 0.200
  Technology: 0.200

😊 Sentiment: positive (score: 0.893)

🔍 Entities Found (7):
  Boston Celtics (ORG) - Companies, agencies, institutions, etc.
  108 (CARDINAL) - Numerals that do not fall under another type
  NBA (ORG) - Companies, agencies, institutions, etc.
  LeBron James (PERSON) - People, including fictional
  35 (CARDINAL) - Numerals that do not fall under another type

💡 Insights:
  ⚠️ Uncertain classification - consider manual review
```

📝 Project Summary and Next Steps

🎯 What You've Accomplished

Congratulations! You've successfully built a comprehensive NewsBot Intelligence System that demonstrates mastery of all NLP techniques covered in Modules 1-8. Let's review what you've achieved:

✅ Module Integration Checklist

- ☑ **Module 1:** Applied NLP to real-world news intelligence
- ☑ **Module 2:** Implemented comprehensive text preprocessing
- ☑ **Module 3:** Used TF-IDF for feature extraction and analysis
- ☑ **Module 4:** Analyzed grammatical patterns with POS tagging
- ☑ **Module 5:** Extracted syntactic relationships with dependency parsing
- ☑ **Module 6:** Performed sentiment and emotion analysis
- ☑ **Module 7:** Built and evaluated text classification models
- ☑ **Module 8:** Implemented Named Entity Recognition

🚀 System Capabilities

Your NewsBot can now:

- Automatically categorize news articles with high accuracy
- Extract key entities (people, organizations, locations, dates, money)
- Analyze sentiment and emotional tone
- Identify linguistic patterns and writing styles
- Generate actionable business insights
- Process new articles through a complete pipeline